A Conditioned Metatheorem for RA-Linearizability of State-Dependent MRDTs

with the tombstone-free RGA as its instance — pen-and-paper proof and mechanization map

2026-07-04 (revised 2026-07-07: mechanization complete)

Abstract

We give a metatheorem: any Mergeable Replicated Data Type whose operations satisfy a fixed set of *conditioned* verification conditions — commutation, a threadable reorder invariant, and a merge–linearization bridge, each *conditioned on a state invariant* Inv *and an applicability predicate* app — is RA-linearizable, once and for all. The conditioning is what admits *state-dependent* datatypes, whose operations commute and merge correctly only on the reachable, sensible states, and which therefore fall outside the unconditioned metatheory. Our flagship instance is the tombstone-free RGA, a replicated list in which deletion physically removes an element and re-anchors its dependents; it discharges every conditioned VC, the hard ones resting on a single invariant ($\mathsf{RecPathFaithful}$ below) that is true by construction. The nine flat production MRDTs discharge the same VCs trivially and inherit RA-linearizability nearly for free. Strong eventual consistency is a corollary. Each step carries its Lean name and kernel status. *As of the revision date the mechanization is complete*: the tombstone-free RGA carries a kernel-checked end-to-end theorem — RA-linearizability up to $\approx$ at every reachable configuration of the replicated store — whose sole assumption is per-step *honest delivery* (each op generated accurately against a causal fold of the events its replica had seen, and delivered born-applicable); everything else, including Lamport clocks and all wellformedness of delivered payloads, is structural or derived (`rga_tombstone_free_ra_linearizable3_eq`).

1 RA-linearizability, and why it must be conditioned

MRDTs. A datatype fixes a state space, an initial state σ_0, an operation set, a transition function $\mathsf{do} : \mathrm{State} \to \mathrm{Op} \to \mathrm{State}$, and a three-way merge $\mathsf{merge} : \mathrm{State}^3 \to \mathrm{State}$ (arguments: the lowest common ancestor and the two branches). Replicas apply local operations with do and reconcile divergent states with merge. Each replica state is a *version*; the versions form a DAG whose leaves are updates (do) and whose internal nodes are merges.

Linearization and canonical states. Fix an event set E with a visibility (causal) order. The *RA order* lo^E orders the pairs a correct sequential execution must not invert: a visible non-commuting pair by visibility, and a concurrent non-commuting pair by the return-value-aware tie-break (the absorber/overwriter clause — ordered by rc unless the later event is already overwritten by a still-later non-commuting one). Two points matter for the proof and were confirmed against the mechanization (`RA_Linearizability.lean`). First, lo^E is *not transitive*: "π respects lo^E" is therefore the elementwise condition $\pi.\mathsf{Pairwise}\,(\lambda a\, b.\ \neg\, \mathsf{lo}^E\, b\, a)$ (`respects`), not sortedness, and lo^E is in general partial — many enumerations respect it, so the canonical state $\sigma^*(E) := \mathsf{fold}\,\sigma_0\,\pi$ is well

defined only up to $\approx$ and only once such folds are shown to converge (Lemma 1). Second, for the *conditioned* metatheorem lo^E is taken over conditioned commutation $\mathsf{commutesOn}$ (`MRDTSig.lean`); the unconditioned lo^E over $\mathsf{commutes}$ (`RA_Linearizability.lean`) is the flat specialization.

Observational equivalence. $\sigma \approx \sigma'$ iff no client operation can distinguish them (equal liveness, payloads, and observable order per element). Representation junk — dead-node residue, internal ordering artefacts — is quotiented away. Convergence and RA-linearizability are always stated up to $\approx$.

Definition 1 (RA-linearizability, per version, up to $\approx$)**.** An MRDT is *RA-linearizable* if, in every well-formed execution, each version v with head state s and delivered event set E_v admits a witnessing linearization: there is a permutation π of E_v (`listPermOf`) that respects lo^E (`respects`) with

$$\mathsf{fold}\,\sigma_0\,\pi \;\approx\; s.$$

This is the mechanized `IsRALinearizable` (`RA_Linearizability.lean`) with one change: structural equality $\mathsf{fold}\,\sigma_0\,\pi = s$ is relaxed to observational $\approx$. The relaxation is exactly what a state-dependent datatype needs — the tombstone-free RGA converges only up to observable behavior (dead-node residue may differ), so equality on the nose is too strong. Given convergence (Lemma 1) the witness form is equivalent to $s \approx \sigma^*(E_v)$; we use the existential form as the definition and the canonical form as a convenience.

Why conditioning is forced. The unconditioned metatheory (below) proves RA-linearizability from VCs that quantify over *all* states. That works for flat datatypes — counters, OR-sets, MVR — whose operations commute and merge on every state. It fails for *state-dependent* datatypes. In the tombstone-free RGA, two concurrent inserts commute only when each one's anchor is faithfully resolvable in the other's presence; an insert under a since-deleted, physically-removed node is not even applicable. Written over all states, the commutation VC is simply false (a mechanized counterexample: `naive_general_swap_false`). The remedy is to condition every VC on a state invariant Inv (what a reachable state guarantees) and an applicability predicate app (when an operation is sensible) — and to prove RA-linearizability from the conditioned VCs. The developer's burden is only to describe sensible operations; the metatheorem does the rest.

2 The conditioned verification conditions

A datatype presents a signature $\langle \mathrm{State}, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{Inv}, \mathsf{app}\rangle$ where Inv is a predicate on states and app a predicate on (operation, state) pairs (Lean: `ConditionedMRDTSig`, `MRDTSig.lean`). It must supply:

VC 1 (Discipline)**.** $\mathsf{Inv}\,\sigma_0$ holds; do preserves Inv on applicable operations; and generation is disciplined: every operation is app and *fresh* at its birth state, with globally unique, monotone ids. (This is the execution model, Lean: `ConditionedConfiguration`, `ConditionedExecutionModel.lean` — kernel-clean, axiom-free.)

VC 2 (Conditioned commutation — the swap witness)**.** For concurrent operations a, b and any state σ with $\mathsf{Inv}\,\sigma$ at which both are applicable in the relevant reorder sense, $\mathsf{do}\,(\mathsf{do}\,\sigma\,a)\,b \approx \mathsf{do}\,(\mathsf{do}\,\sigma\,b)\,a$.

VC 3 (Threadable reorder invariant)**.** An auxiliary invariant J on (state, pending-event) that (i) certifies applicability/the swap precondition of VC 2 at reorder states, (ii) holds at the enabling

fold of each event, and (iii) is preserved by each reorder step. Only the *enabled* events — those whose causal past is folded — need be certified; that scope is exactly what a linearization repair ever transposes.

VC 4 (Merge–linearization bridge — the conditioned Join Lemma)**.** For a reachable LCA state l and branches a, b obtained from l by disjoint concurrent event lists, $\mathsf{merge}\, l\, a\, b \approx \mathsf{fold}\, l\, \pi$ for a lo^E-respecting enumeration π of the branch events.

For flat datatypes $\mathsf{Inv} = \mathsf{app} = \mathsf{true}$, VC 2 degenerates to unconditioned commutation, VC 3 to $J = \mathsf{true}$, and VC 4 to the ordinary Join Lemma — recovering the unconditioned methodology.

3 The metatheorem

Theorem 1 (Soundness — conditioned VCs ⇒ RA-linearizability)**.** *Any MRDT satisfying VCs 1–4 is RA-linearizable (Definition 1).*

The proof is generic — it never inspects a particular datatype — and factors through two lemmas, each proved over the abstract signature with $\approx$ an arbitrary congruence for do and merge.

Remark 1 (Route actually taken). The convergence lemma below is stated via the swap-based VC 3 for exposition. That route is *machine-refuted* for the tombstone-free RGA; convergence is mechanized by direct canonical-state characterization instead. See Section 5.

Lemma 1 (Convergence)**.** *VCs 1, 2, 3 imply that all lo^E-respecting admissible enumerations of a backward-closed reachable E fold from σ_0 to $\approx$-equal states; hence $\sigma^*(E)$ is well defined up to $\approx$.*

Proof. Any two lo^E-respecting enumerations of E are related by repeatedly transposing adjacent lo^E-incomparable events. Because lo^E is non-transitive, "respecting" is the elementwise `Pairwise` condition rather than sortedness, so this order-repair is more delicate than for a total order — it is exactly what the generic engine `eq_convergence` carries out, and we appeal to it rather than re-derive it. Each transposition preserves the fold up to $\approx$ by VC 2, provided the swap precondition holds at the transposition point; VC 3 supplies it along the whole enumeration (base at each event's enabling fold, preserved by each step, scoped to the enabled events — the only ones the repair transposes). The engine consumes only that $\approx$ is an equivalence, do is $\approx$-congruent, and a swap oracle of the shape VC 2+3 provides (Lean: `eq_convergence`, kernel-clean). □

Lemma 2 (Canonical adequacy)**.** *VC 4 and Lemma 1 imply that every reachable version state is $\approx\ \sigma^*$ of its delivered event set.*

Proof. Induction over the version DAG. Initial: the empty fold. Update node: appends one delivered operation to a canonical fold (definitional). Merge node with LCA l and branches a, b: by hypothesis $a \approx \sigma^*(E_a)$ and $b \approx \sigma^*(E_b)$, and $l \approx \sigma^*(E_l)$. First rewrite a, b to literal canonical folds using $\approx$-congruence of merge (so VC 4, stated for branch folds, applies); then VC 4 gives $\mathsf{merge}\, l\, a\, b \approx \mathsf{fold}\, l\, \pi$ for a lo^E-respecting π of the branch events $E_a \cup E_b$. Now E_l causally precedes those branch events (they were generated on states derived from l), so a lo^E-respecting enumeration of $E_l \cup E_a \cup E_b$ factors as (an lo^E-enumeration of E_l) followed by π; hence $\mathsf{fold}\, l\, \pi = \mathsf{fold}\, (\sigma^*(E_l))\, \pi \approx \sigma^*(E_l \cup E_a \cup E_b)$ by Lemma 1 (the engine is start-state generic). Backward closure keeps every enumeration admissible. □

Proof of Theorem 1. Immediate from Lemma 2: each version state $\approx \sigma^*(E_v)$, which is Definition 1. (Unconditioned template: `ra_linearizable3_of_joinC` plus the closure-indexed adequacy bridge, already kernel-clean with nine instances; the conditioned metatheorem generalizes it by carrying $\mathsf{Inv}/\mathsf{app}$ through the two lemmas.) □

Corollary 1 (Strong eventual consistency)**.** *Two versions with the same delivered set E have $\approx$ states: both $\approx \sigma^*(E)$ by Theorem 1, hence $\approx$ each other.*

4 The tombstone-free RGA discharges the conditioned VCs

The RGA is a forest of identified nodes; do inserts a fresh node under a resolved anchor or physically deletes a node and rehomes its children; merge keeps OR-set survivors and re-anchors each by climbing to the nearest surviving ancestor. resolve $\sigma\, p$ returns the first live id on a recorded path p; anc $\sigma\, x$ is x's current parent. Take Inv := wf $\wedge$ id_mono $\wedge$ (root not reinserted) and app := accurate $\wedge$ fresh. We discharge each VC; the two hard ones reduce to one invariant.

Definition 2 (Recorded-path faithfulness RecPathFaithful)**.** RecPathFaithful $a\, \sigma$: the recorded path rec a is a *live subchain of genuine ancestors* of a's target in σ — true anc-ancestors, nearest first, consecutive pairs real parent edges, and exactly the chain live at a's generation. (This is `accurate/HistFaithful` promoted to a reachability invariant.)

Lemma 3 (RecPathFaithful is a reachability invariant)**.** *RecPathFaithful $a\, \sigma$ holds at every reachable σ in which a is applied.*

Proof. Induction on the operations building σ. *Birth:* do sets rec a = resolve-path at generation = the live-ancestor chain, so RecPathFaithful holds (`chainFaithful_of_histFaithful`). *Later* Ins *with fresh id f:* attaches f as a descendant — no reparenting, no liveness change for prior events' chains. f cannot appear spuriously in any rec a: its members are older ids $< a.\mathsf{id} \leq$ every id extant at a's birth, whereas f exceeds all of them, so $f \notin$ rec a definitionally. The resurrection/clash regime (`chainFaithful_not_preserved_under_clash_ins`) is therefore *unreachable* by freshness (`fresh_not_mem_recList`, `clash_recList_excluded`), not excluded by a side condition. *Later* Del x*:* flips x's liveness only; rec a's entries stay genuine ancestors, one possibly now dead — still a live subchain (`chainFaithful_doDel_faithful`). □

Lemma 4 (Key Lemma: resolution over a live subchain)**.** *For any live subchain of genuine ancestors pre of x, in every reachable σ,* resolve $\sigma\, pre$ = anc $\sigma\, x$*. (Lean: `subchain_resolve`, `RGA_SubchainResolve.lean`, kernel-clean.)*

Proof. resolve returns the first entry live in σ; each entry is a genuine ancestor, so this is the nearest live ancestor among pre. It is the nearest overall: no live ancestor $c \notin pre$ is nearer than that first-live entry — (a) if c was live at capture it was in the captured live chain, so $c \in pre$ (and any pre-entry nearer than the first-live one is dead in σ, hence not c); (b) if c was dead at capture it is still dead (physical, tombstone-free deletion is permanent); (c) an event inserted after capture attaches as a descendant, never a new ancestor of the pre-existing x. The mechanization absorbs these three cases into one preserved invariant LiveChain $\sigma\, x\, pre$:= (root unstored) $\wedge$ live $\sigma\, x \wedge$ IsAncPath $\sigma\, x$ (pre.filter live), which holds at capture, is preserved by each insert (`isAncPath_upd`) and delete (`isAncPath_surgery`, chain splicing across a deleted entry), and yields the conclusion. Case (b) is discharged by an explicit hypothesis $t \notin pre$ on inserts (`okStep`) — the mechanized form of "no resurrection," itself a consequence of monotone allocation, every pre-entry sitting below x's id. This replaces the earlier full-chain lemma `hres_of_lchain`, which failed on the proper subchains rehoming produces. □

Proposition 1 (RGA discharge)**.** *The RGA satisfies VCs 1–4. (The convergence VC 3 is discharged not by the swap route sketched here but by canonical-state characterization, Section 5; the swap-route sketch below is superseded.)*

Proof. VC 1 (discipline): Inv preservation and the generation model are `ConditionedConfiguration` instantiated at the RGA, with the RGA's do-invariance lemmas; `conditioned_premises` supplies the timestamp/freshness facts. *VC 2* (swap): `general_swap_bothFaithful` / `eqSwap_of_bothFaithful` — concurrent operations commute up to $\approx$ when both are *faithful* (neither need be exact/accurate), kernel-clean. *VC 3* (reorder invariant): $J :=$ ChainFaithful on the enabled scope, with steps `chainFaithful_incompStep/incompFold`; its *base* — each event faithful at its enabling fold — is exactly RecPathFaithful (Lemma 3) projected through the Key Lemma (Lemma 4): the residual goal $\mathsf{resolve}\,\sigma\,((\mathsf{rec}\,a) \setminus t') = anch'$ when an ancestor t' becomes leftmost live is the Key Lemma on the remaining subchain. *VC 4* (Join): per-id extensional match (`eq_merge2_of_branchInv2`); the anchor-coincidence clause is the single-sided I_4 (`eq_merge_single`, kernel-clean) and, cross-branch, the Key Lemma directly (`branchInv_doDel_crossBranch`, kernel-clean); order-independence is Lemma 1 at start state l. (In the final assembly this per-id BranchInv threading is itself superseded by the birth-anchor bridges of Section 5, which need no invariant threading at all.) Both hard VCs (3 base, 4 cross-branch) thus rest on RecPathFaithful and the Key Lemma — proved by construction from freshness and permanence. □

Corollary 2. *The tombstone-free RGA is RA-linearizable (Theorem 1 + Proposition 1), and strongly eventually consistent (Corollary 1). To our knowledge this is the first mechanized RA-linearizability / SEC result for an RGA with physical, tombstone-free deletion (cf. Gomes et al., OOPSLA'17, for the tombstoned RGA).*

Remark 2 (Other instances). The nine flat production MRDTs (counter, OR-set, MVR, …) take Inv = app = true: VC 2 is their unconditioned commutation, VC 3 is $J =$ true, and VC 4 is the ordinary Join Lemma — already discharged (`ra_linearizable3_of_joinC`). They inherit RA-linearizability from the same Theorem 1. The RGA is the instance that exercises every conditioned mechanism; the framework supplies the generality, the RGA the novelty.

5 How it was actually mechanized (and two dead ends)

The exposition above (Sections 2–4) is the *intended* route, and it correctly frames the contribution: conditioned VCs ⇒ RA-linearizability, RGA as instance. But the mechanization did *not* follow the swap-based convergence of VC 3. That route was tried and **machine-refuted twice**; convergence was then proved by a different technique. We record the actual route honestly — the refutations are themselves findings about why tombstone-free deletion is hard.

Two machine-checked refutations of the swap route. The reorder-invariant convergence (VC 3, "thread faithfulness through adjacent swaps") fails for the tombstone-free RGA because *rehoming makes an event's recorded chain time-relative*: correct at generation, wrong at reordered intermediate states.

- lo^E **is not transitive** (`RGA_DepComp_Gate`, kernel-clean refutation). Reordering dependencies-contiguous-first needs transitivity of the linearization order; a physical delete severs an operation's applicability-dependence on an ancestor ($b \to a \to o$ but $\neg(b \to o)$ after a's target is deleted). Witness: $l = 0 \leftarrow 1 \leftarrow 2 \leftarrow 3$, delete node 1, delete node 2.
- **In-place threading fails at the ancestor-insert step** (`RGA_SimulInduction2`, kernel-clean refutation). Even threading without reordering, an event's recorded chain can be *ahead* of the state (it anticipates a delete not yet folded), so ChainFaithful of the recorded chain is false at that intermediate fold although it heals later.

Both say the same thing: the linearize-and-swap method, which needs faithfulness at every *intermediate* reordered state, is the wrong vehicle. This is exactly what tombstones prevent (Gomes et al. keep deleted nodes so dependencies never rewire); the tombstone-free RGA rewires them.

The technique that works: direct canonical-state characterization. Instead of swaps, characterize the fold state as a pure function of the applied event *set* — the update analog of the merge bridge (which always worked). For a finite applied set F, define $\mathsf{survivors}\,F$ (OR-set survival) and $\mathsf{canonAnc}\,F\,k$ (climb k's recorded chain to the nearest survivor of F); let $\mathsf{CanonMatch}\,F\,s$ say s matches this per id.

Lemma 5 (Canonical fold)**.** *Along any* lo^E*-respecting enumeration* π*,* $\mathsf{CanonMatch}\,(\pi\text{-}\mathit{applied})\,(\mathsf{fold}\,\sigma_0\,\pi)$ *(Lean:* `canon_fold`*, kernel-clean). Two folds of the same* E *then both match* $\mathsf{CanonMatch}\,E$*, hence are* $\approx$ *— convergence with no swaps and no intermediate-state faithfulness (*`RGA_update_convergence_canon`*; discharged from the per-event generation discipline* GenDisc2C *in* `RGA_update_convergence_final`*).*

The invariant is over the *applied* set, so $\mathsf{canonAnc}\,F$ never anticipates a future delete — exactly why it succeeds where the swap route (which reasons about not-yet-applied events) fails. The Key Lemma (Lemma 4) survives unchanged as the per-id climb-vs-resolve coincidence; RecPathFaithful is subsumed by CanonMatch's per-survivor `LiveChain`.

The metatheorem as a generic $\approx$-quotient functor. Theorem 1 is mechanized generically over `ConditionedMRDTSig` (`RA_linearizable_up_to_eq`, `GenericEqQuotient`, kernel-clean). It quotients the state by $\approx$ over the Inv-subtype and transfers the datatype's $\approx$-Join to the structural-$=$ template `ra_linearizable3_of_joinC` by `Quotient.sound`; the readback is over *raw* fold, matching the RGA's convergence. The datatype supplies five VCs: EqEquiv, InvPres, CongVC (update/merge/query $\approx$-congruent on Inv), InvInvVC, and the $\approx$-Join EqJoinLemma3C. Two subtleties the mechanization forced, each machine-verified:

- **The merge $\approx$-congruence is false on the full state type** (`merge_eq_congr_l_fails`, axiom-free), true on the reachable subfamily (wf $\wedge$ id_mono $\wedge$ forest): the LCA-climb reads $\mathsf{anc}\,l$ off $\mathsf{domain}\,l$, where $\approx$ is silent. Hence the Inv-conditioning of CongVC is load-bearing, and the quotient is over the Inv-subtype (`merge_eq_congr_inv`).
- **Invariant preservation needs well-formedness, not applicability.** InvPres cannot demand unconditional preservation (do at a bad id breaks the invariant), nor full app (which the execution model does not guarantee — `Step3.apply` applies with only timestamp freshness). The honest condition is a datatype predicate WfOp (for the RGA: fresh id for Ins; $\mathsf{resolve}\,s\,p \neq x$ for $\mathsf{Del}\,p\,x$), preserved by the real do (`rgaInv_doOp_fresh`, accuracy *removed*) and holding along reachable folds (WfOpReachable). A totality guard on the lifted step is provably transparent on the execution domain (applySeqW = applySeq); the exposed theorem is over the real do.

The completed assembly. Everything after the quotient functor was, at the first writing of this note, "bounded assembly." Carrying it out surfaced three further machine-checked refutations that reshaped the target and the witness discipline; we record them because each is a fact about tombstone-free deletion, not about our proof.

- **At a merge union, applicability-tolerant witnesses do not exist.** An LCA-first enumeration pre-applies one branch's anchor-killing deletes before replaying the other branch's inserts, staling their recorded paths — so even *no-op-feasible* witnesses (each step applicable

or a no-op) are unsatisfiable from the initial state (`RGA_HEnum_Refutation`). Worse, on a seven-event two-branch execution *no* born-applicable enumeration of the union replays both branches' recorded observations: rehoming makes the *order of concurrent deletes* observable in survivors' stored parents, and the branches observe it differently. Consequently the linearization target itself must be the *raw* `do`-fold, up to $\approx$ — any guarded or feasibility-filtered fold is unsatisfiable at merge unions (`IsRALinearizable3Eq`, `GoodConfig3H.lean`).

- **The witness discipline is the engine's own, plus payload honesty.** Each version carries, existentially, an enumeration satisfying the canonical engine's per-step discipline (`CanonFoldOK`: fresh nonzero ids, no id reuse, live recorded chains) *and* payload honesty (`HonestPayloads`: delete targets and recorded-chain entries are root-or-inserted). The second clause is forced by a counterexample: the fold discipline alone admits a dead-target delete (a no-op), and a later insert reusing that recorded dead id breaks the no-id-reuse clauses — honesty of *payloads* is set-level, hence permutation-invariant, and it is exactly what extension at a fresh apply needs (`rga_hHext_discharged`). The discipline *joins* at merges with the union witness being the LCA-then-delta concatenation $\rho_0 +\!\!+ \pi_0$ itself: no reordering, no swaps, anywhere (`rgaJoinH_of_canon`).
- **The generation discipline is derived, not assumed.** Every event is accurate at the fold of its *transitive dependencies* (`GenDisc2C`), provable from *born accuracy* — the op was generated against a causal fold of the events its replica had seen (`genDisc2C_of_born`). Merge canonicity then reduces to three leaves, all discharged: σ_0 id-monotonicity with *no fold induction* (the stored anchor is `canonAnc` of the recorded chain, whose entries are dependencies, which are Lamport-below — `rga_Hdec_discharged`); a per-id causal set-algebra whose only non-trivial clauses are dependency-fold provenance (`rga_hcaus_discharged`); and per-survivor *birth-anchor bridges* reducing the merged forest's anchors to record coherence at dependency folds, where the generation discipline makes every recorded chain live (`rga_hbridge_discharged`, `hin_of_genDisc`).

The residual, quantified once over the execution, is a single per-step assumption (`HonestDelivery`): *born accuracy* plus *born-applicable delivery*. It is genuinely irreducible — it is the generation discipline tombstone-freedom forces — and it is also all there is: Lamport clocks, timestamp uniqueness and visibility-support are structural fields of the configuration (dishonest-clock executions are unrepresentable); nonzero insert ids and nonzero delete targets follow from the delivered op's own wellformedness at a representative of its head class; nonzero delete *timestamps* are Lamport-derived from the target's insert. The end-to-end theorem (`rga_RA_linearizable_honest`, mainline entry point `rga_tombstone_free_ra_linearizable3_eq` in `Sal/MRDTs/Metatheory/RGA_TombstoneFree_RA_Lin.lean`) closes Proposition 1 and the corollary unconditionally modulo that assumption.

6 Mechanization status

Kernel-clean means axioms $\subseteq$ {`propext`, `Classical.choice`, `Quot.sound`}, no `sorryAx`/`native_decide`/`ofReduceBool`. Status as of the revision date: *every row is proved.*

Component	Lean	Status
Unconditioned template (Inv, structural =)	`ra_linearizable3_of_joinC`	proved (9 instances)
Generic conditioned metatheorem (Thm 1, ≈-quotient)	`RA_linearizable_up_to_eq`	**proved**
Execution model (VC 1)	`ConditionedConfiguration`	proved (axiom-free)
Key Lemma (4)	`subchain_resolve`	proved
Canonical fold / update convergence (5)	`canon_fold`, `RGA_update_convergence_final`	proved
Merge single-sided (I_4) / cross-branch	`eq_merge_single`, `branchInv_doDel_crossBranch_sub`	proved
Cross-forest bridge	`resolve_climb_start`	proved
Merge ≈-congruence on Inv (VC_{cong})	`merge_eq_congr_inv`	proved
WfOp preservation (accuracy-free)	`rgaInv_doOp_fresh`	proved
RGA VC bundle (`EqEquiv`,`InvInvVC`,update-cong)	`RGA_VCPackage`	proved
Raw-≈ target + H-witness layer	`IsRALinearizable3Eq`, `RA_linearizable_up_to_eq_H`	proved
H-join (union witness $\rho_0 +\!\!+ \pi_0$)	`rgaJoinH_of_canon`	proved
Generation discipline from born accuracy	`genDisc2C_of_born`	proved
Delta enumeration (hEnum, K1)	`rga_hEnum_discharged`	proved
Merge-canonicity bundle (Hdec, hcaus, hbridge)	`rga_hMergeInputs_discharged`	proved
Record coherence at dependency folds	`hin_of_genDisc`	proved
Discipline extension at applies (hHext)	`rga_hHext_discharged`	proved
Honest residual (hHon, hBA from `HonestDelivery`)	`rga_hHon_discharged`, `honCore_of_reach`	proved
End-to-end RGA theorem	`rga_RA_linearizable_honest`, `rga_tombstone_free_ra_linearizable3_eq`	**proved**
Flat MRDTs (MVR, AW-PQ) via identity quotient	—	future work

Reading. The chain is closed. The two genuinely hard problems — convergence of the tombstone-free RGA under rehoming, and making the metatheorem generic over ≈ — are proved by canonical-state characterization (after the swap route was refuted twice) and by the ≈-quotient functor; the assembly that this note's first version listed as residual is done, and doing it forced the further findings recorded above (the raw-≈ target, payload honesty, the birth-anchor bridges). The theorem is over the real do, at every reachable configuration, and its entire trust statement is one per-step assumption — `HonestDelivery`: ops are generated accurately against a causally-delivered state and delivered born-applicable. Property-based testing had corroborated every step in advance (600 merge-vs-fold triples through the hard corners including cross-branch anchor and shared delete, zero counterexamples); the kernel now certifies them. Remaining future work is optional breadth, not depth: route the flat MRDTs (MVR, AW-PQ) through the same five VCs with the identity quotient, and give the RGA an ≈-collapsing "eq variant" whose observational equivalence is structural equality.